

Template • Brief per nuovo plugin CMS-R4 (copia-incolla)

Questo file è pensato per essere **incollato come primo messaggio** quando vuoi che io generi un nuovo plugin per il tuo CMS-R4. Compila i campi tra parentesi `[]` e mantieni il resto invariato. Sotto trovi anche **scheletri di codice** (manifest, JS/CSS) e una **checklist** di rilascio.

PROMPT • COPIA & INCOLLA

Incolla tutto il blocco qui sotto nel nuovo chat e compila i campi

CONTESTO

- Progetto: CMS-R4Software (Laravel 12.x / PHP 8.2+) con PluginManager + Page Builder.
- Plugin: front-only, asset pubblici in /public/plugins/{slug}, ****nessun ESM**** (niente type="module").
- Il renderer cerca ``window.BuilderPlugins["plugin:{slug}"]`` e usa ``mount(el, data)`` o ``renderView(data)``.
- In admin, se presente ``renderEditor(sec, block)``, il builder usa quello per il pannello impostazioni.
- Asset front dichiarati in manifest ``assets.front`` (CSS/JS) + (opz.) ``admin_entry``.

OBIETTIVO

- Costruisci un nuovo plugin.

METADATI PLUGIN

- Name: [NOME PLUGIN]
- Slug: [slug-kebab]
- Version: [1.0.0]
- Author: [R4Software]

BLOCCHI BUILDER

- Tipo blocco: plugin:[slug-kebab]
- Label: [Etichetta visibile]
- Icon (Bootstrap Icons): [bi-...]
- Colonne iniziali: [1..12]

DATA MODEL BLOCCO (serializzato in block.data)

- Struttura: [descrivi i campi, es: {items:[{src,alt,url}], height:number, gap:number, speed:number, ...}]
- Valori default: [elenco dei default]

UI ADMIN (renderEditor)

- Campi e controlli richiesti: [elenco UI: input number, toggle, picker, add/remove items, ecc.]

- Regole/validazioni: [vincoli]

FRONTEND (mount/renderView)

- Layout desiderato: [griglia/carousel/lista...]
- Comportamento: [animazioni, autoplay, responsive]
- Accessibilità: [alt obbligatori, aria-, focus]

ASSET

- Necessari: [view.css, view.js, admin.js]
- Dipendenze: [nessuna / libreria X (se già presente)]

DELIVERABLES

- Genera: plugin.json, public/view.js, public/view.css, public/admin.js.
- Codice: IIFE, usa `window.BuilderPlugins`, nessun import ESM.
- CSS scoped con prefisso: .[sigla breve]

ACCETTAZIONE

- Il CSS deve posizionare gli elementi **in orizzontale** dove previsto (es. `display:flex` sulla track) per evitare layout "a colonna".
- I file in `assets.front` devono puntare a percorsi relativi alla cartella `public/` del plugin.
- Prevedi `document.dispatchEvent(new Event('r4lc:refresh'))` al termine dei file JS.

TEST RAPIDI

- Fornisci un JSON di esempio per `data` e un HTML di test minimal per anteprima.

OUTPUT

- Restituisci i file completi (codice pronto), più eventuali note di integrazione.

SCHELETRI DI CODICE (riutilizzabili)

1) plugin.json (template)

```
{
  "slug": "[slug-kebab]",
  "name": "[NOME PLUGIN]",
  "version": "[1.0.0]",
  "author": "R4Software",
  "blocks": [
    { "type": "plugin:[slug-kebab]", "label": "[Etichetta]", "icon": "[bi-icon]", "add_columns": [12] },
  ],
  "assets": { "front": ["view.css", "view.js"] },
}
```

```

    "admin_entry": "admin.js"
  }

```

2) public/view.js (IIFE, no ESM)

```

(function(){
  const TYPE = 'plugin:[slug-kebab]';
  const NS = (window.BuilderPlugins = window.BuilderPlugins || {});

  function renderHTML(data){
    const d = Object.assign({ items:[], height:72, gap:24, speed:30 }, data|| {});
    const items = (d.items||[]).map(i => `
      <div class="[prefix]-item"></div>`
    ).join('');
    return `
      <div class="[prefix]" style="--h:${d.height}px;--gap:${d.gap}px;--speed:${d.speed}s">
        <div class="[prefix]-track">${items}</div>
      </div>`;
  }

  NS[TYPE] = {
    mount(el, data){ el.innerHTML = renderHTML(data); el.querySelector('.pb-plugin--placeholder')?.classList.add('d-none'); },
    renderView(data){ return renderHTML(data); }
  };

  document.dispatchEvent(new Event('r4lc:refresh'));
})();

```

3) public/admin.js (pannello impostazioni)

```

(function(){
  const TYPE = 'plugin:[slug-kebab]';
  const NS = (window.BuilderPlugins = window.BuilderPlugins || {});
  NS[TYPE] = NS[TYPE] || {};

  NS[TYPE].renderEditor = function(sec, block){
    const d = block.data || (block.data = { items:[], height:72, gap:24, speed:30 });
    return `
      <div class="row g-2">
        <div class="col-4">
          <label class="small">Altezza (px)</label>
          <input type="number" class="form-control form-control-sm" value="${d.height}"
            oninput="updatePluginField('${sec.id}','$

```

```

{block.id}', 'data.height', parseInt(this.value||72,10));">
    </div>
    <div class="col-4">
        <label class="small">Gap (px)</label>
        <input type="number" class="form-control form-control-sm" value="$
{d.gap}"
            oninput="updatePluginField('${sec.id}','$
{block.id}', 'data.gap', parseInt(this.value||24,10));">
    </div>
    <div class="col-4">
        <label class="small">Velocità (s)</label>
        <input type="number" class="form-control form-control-sm" value="$
{d.speed}"
            oninput="updatePluginField('${sec.id}','$
{block.id}', 'data.speed', parseInt(this.value||30,10));">
    </div>
</div>
<div class="mt-2">
    <div class="d-flex justify-content-between align-items-center mb-1">
        <label class="small mb-0">Elementi</label>
        <button type="button" class="btn btn-sm btn-outline-primary"
            onclick="(function(){ const b=findBlockById('${
{block.id}').blk; (b.data.items=b.data.items||[]).push({src:'',alt:''});
renderBuilder(); })())">+ Aggiungi</button>
    </div>
    ${ (d.items||[]).map((it,i)=>`
        <div class="row g-2 align-items-center mb-1">
            <div class="col"><input class="form-control form-control-sm"
placeholder="URL immagine" value="${it.src||''}"
                oninput="updatePluginField('${sec.id}','$
{block.id}', 'data.items.${i}.src', this.value)"></div>
            <div class="col"><input class="form-control form-control-sm"
placeholder="Alt" value="${it.alt||''}"
                oninput="updatePluginField('${sec.id}','$
{block.id}', 'data.items.${i}.alt', this.value)"></div>
            <div class="col-auto">
                <button type="button" class="btn btn-sm btn-outline-danger"
                    onclick="(function(){ const b=findBlockById('${
{block.id}').blk; b.data.items.splice(${i},1); renderBuilder(); })())">🗑️</
button>
            </div>
        </div>`
        </div>`).join('')}
    </div>`;
};

document.dispatchEvent(new Event('plugins:ready'));
})();

```

4) `public/view.css` (base orizzontale, anti-colonna)

```
/* Prefix consigliato: .[prefix] */
.[prefix]{ position:relative; overflow:hidden; --h:72px; --gap:24px; --speed:
30s; }
.[prefix]-track{ display:flex; align-items:center; gap:var(--gap); min-
height:var(--h); }
.[prefix]-item{ flex:0 0 auto; display:flex; align-items:center; justify-
content:center; height:var(--h); }
.[prefix]-item img{ max-height:100%; width:auto; object-fit:contain;
display:block; }
```

PACKAGING (ZIP)

1) Struttura cartelle:

```
[slug-kebab]/
├─ plugin.json
├─ public/
│   └─ view.js
│   └─ view.css
└─ admin.js
```

2) Comprimi il **contenuto** della cartella `[slug-kebab]` in `[slug-kebab].zip`. 3) In Admin → Plugin: carica ZIP, abilita.

CHECKLIST DI RILASCIO

- [] `slug` valido (`^[a-z0-9-]+$`).
- [] `blocks[].type` = `plugin:[slug]`, label/icon/colonne ok.
- [] `assets.front` punta a `view.css`, `view.js` dentro `public/`.
- [] JS **IIFE** che registra `window.BuilderPlugins[TYPE]` (no ESM).
- [] `renderEditor` presente se servono controlli custom.
- [] CSS con `display:flex` sulla track per layout orizzontale.
- [] `document.dispatchEvent(new Event('r4lc:refresh'))` in `view.js` e `plugins:ready` in `admin.js`.
- [] JSON di esempio `data` fornito per test rapido.

DATA EXAMPLE (facoltativo da incollare nel builder)

```
{
  "items": [
    { "src": "/media/logo-1.svg", "alt": "Logo 1" },
```

```
    { "src": "/media/logo-2.svg", "alt": "Logo 2" },  
    { "src": "/media/logo-3.svg", "alt": "Logo 3" }  
  ],  
  "height": 72,  
  "gap": 24,  
  "speed": 30  
}
```

Suggerimento: conserva questo file come “modello” e duplicalo per ogni nuovo plugin. Così, quando mi scrivi, basta incollare il blocco PROMPT compilato e io seguirò queste convenzioni senza dover rispiegare il sistema.